

TUGAS PENGKODEAN DAN PEMORGRAMAN
Dataset Garment



Nama : Amanda Febriana Putri
NIM : 12030123130155
Mata Kuliah : Pengkodean dan Pemrograman
Kelas : E
Dosen Pengampu : Dr. Totok Dewayanto, S.E.,M.Si.,Akt.

PROGRAM STUDI AKUNTANSI
FAKULTAS EKONOMIKA DAN BISNIS
UNIVERSITAS DIPONEGORO
202

Sebutkan urutan data analitik secara bertahap, dan buat tahapan tersebut untuk kepentingan pembuatan kueri sql berbasis google big query

Berikut adalah urutan data analitik secara bertahap yang relevan untuk pembuatan kueri SQL berbasis Google BigQuery, beserta penjelasan tahapan yang dioptimalkan untuk konteks ini:

1. Define Objective (Menentukan Tujuan Analisis)

- Identifikasi pertanyaan bisnis atau tujuan analitik, misalnya: "Berapa total penjualan per kategori produk dalam 3 bulan terakhir?"
- Tentukan metrik (KPI) dan dimensi yang relevan (contoh: total_sales, product_category, date).
- Dalam BigQuery: Pastikan Anda memahami dataset dan skema tabel yang akan digunakan (misalnya, nama tabel, kolom, tipe data).

2. Data Exploration (Eksplorasi Data)

- Pelajari struktur dataset di BigQuery menggunakan perintah seperti:

sql

Copy

```
SELECT * FROM `project.dataset.table` LIMIT 10;
```

- Identifikasi kolom yang relevan, tipe data (STRING, INT64, TIMESTAMP, dll.), dan potensi nilai null/missing data dengan:

sql

Copy

```
SELECT column_name, COUNT(*) AS total, COUNTIF(column_name IS NULL) AS null_count
```

```
FROM `project.dataset.table`
```

```
GROUP BY column_name;
```

- Periksa distribusi data, misalnya:

sql

Copy

```
SELECT product_category, COUNT(*) AS count
```

```
FROM `project.dataset.table`
```

```
GROUP BY product_category
```

```
ORDER BY count DESC;
```

3. Data Preparation (Persiapan Data)

- Tentukan filter, join, atau transformasi yang diperlukan. Misalnya, jika data penjualan ada di tabel terpisah, lakukan join:

sql

Copy

```
SELECT a.order_id, a.date, b.product_category, a.sale_amount
```

```
FROM `project.dataset.orders` a
```

```
JOIN `project.dataset.products` b
```

```
ON a.product_id = b.product_id;
```

- Bersihkan data, seperti menangani nilai null atau format tanggal:

sql

Copy

```
SELECT
```

```
    order_id,
```

```
    CAST(date AS DATE) AS formatted_date,
```

```
    COALESCE(sale_amount, 0) AS sale_amount
```

```
FROM `project.dataset.orders`;
```

- Buat CTE (Common Table Expression) untuk mempermudah pipeline:

sql

Copy

```
WITH cleaned_data AS (
```

```

SELECT order_id, CAST(date AS DATE) AS date, sale_amount

FROM `project.dataset.orders`

WHERE date IS NOT NULL

)

SELECT * FROM cleaned_data;

```

4. Query Development (Pengembangan Kueri)

- o Tulis kueri utama berdasarkan tujuan. Misalnya, untuk menghitung total penjualan per kategori:

sql

Copy

```

SELECT

    b.product_category,

    SUM(a.sale_amount) AS total_sales,

    COUNT(DISTINCT a.order_id) AS total_orders

FROM `project.dataset.orders` a

JOIN `project.dataset.products` b

ON a.product_id = b.product_id

WHERE a.date >= DATE_SUB(CURRENT_DATE(), INTERVAL 3 MONTH)

GROUP BY b.product_category

ORDER BY total_sales DESC;

```

- o Gunakan fungsi BigQuery seperti DATE_TRUNC, APPROX_COUNT_DISTINCT, atau UNNEST untuk kasus kompleks seperti analisis array atau pengelompokan waktu:

sql

Copy

```
SELECT

    DATE_TRUNC(a.date, MONTH) AS month,

    SUM(a.sale_amount) AS total_sales

FROM `project.dataset.orders` a

GROUP BY month

ORDER BY month;
```

5. Query Optimization (Optimasi Kueri)

- Kurangi pemrosesan data dengan memfilter awal menggunakan WHERE sebelum JOIN atau GROUP BY.
- Gunakan partisi atau clustering pada tabel BigQuery untuk mempercepat kueri:

sql

Copy

```
SELECT *

FROM `project.dataset.orders`

WHERE date >= '2025-01-01'

AND date < '2025-04-01';
```

- Hindari SELECT *; sebutkan kolom spesifik untuk mengurangi penggunaan slot.
- Gunakan EXPLAIN untuk menganalisis rencana eksekusi kueri:

sql

Copy

```
EXPLAIN SELECT ...;
```

6. Validation and Testing (Validasi dan Pengujian)

- Validasi hasil kueri dengan memeriksa apakah output sesuai dengan ekspektasi. Misalnya, bandingkan total penjualan dengan laporan lain.
- Uji kueri dengan subset data kecil:

sql

Copy

```
SELECT *  
  
FROM `project.dataset.orders`  
  
WHERE date = '2025-01-01'  
  
LIMIT 100;
```

- Periksa konsistensi data, seperti memastikan tidak ada duplikasi:

sql

Copy

```
SELECT order_id, COUNT(*) AS count  
  
FROM `project.dataset.orders`  
  
GROUP BY order_id  
  
HAVING count > 1;
```

7. Visualization and Reporting (Visualisasi dan Pelaporan)

- Ekspor hasil kueri ke alat visualisasi seperti Google Data Studio atau simpan sebagai tabel baru:

sql

Copy

```
CREATE OR REPLACE TABLE `project.dataset.sales_summary`  
  
AS  
  
SELECT
```

```

        b.product_category,

        SUM(a.sale_amount) AS total_sales

FROM `project.dataset.orders` a

JOIN `project.dataset.products` b

ON a.product_id = b.product_id

GROUP BY b.product_category;

```

- Jadwalkan kueri untuk pembaruan otomatis menggunakan BigQuery Scheduled Queries jika diperlukan.

8. Iterate and Refine (Iterasi dan Penyempurnaan)

- Kumpulkan feedback dari pemangku kepentingan untuk menyesuaikan kueri. Misalnya, tambahkan dimensi baru seperti wilayah:

sql

Copy

```

SELECT

        b.product_category,

        a.region,

        SUM(a.sale_amount) AS total_sales

FROM `project.dataset.orders` a

JOIN `project.dataset.products` b

ON a.product_id = b.product_id

GROUP BY b.product_category, a.region;

```

- Optimasi lebih lanjut berdasarkan performa atau kebutuhan baru.

Catatan Penting untuk BigQuery:

- Manfaatkan fitur seperti tabel yang dipartisi berdasarkan tanggal (PARTITION BY date) atau clustering untuk efisiensi.
- Gunakan APPROX_* functions (misalnya, APPROX_COUNT_DISTINCT) untuk estimasi cepat pada dataset besar.
- Pastikan izin akses dataset sudah diatur di Google Cloud IAM.
- Pantau biaya pemrosesan kueri dengan BigQuery Pricing Calculator atau laporan penggunaan slot.

Dengan mengikuti tahapan ini, Anda dapat membangun kueri SQL di BigQuery yang efisien, akurat, dan sesuai dengan kebutuhan analitik. Jika Anda memiliki kasus spesifik, silakan beri detail untuk contoh kueri yang lebih terfokus!

File CSV yang diberikan oleh Grok

1. Inventory

inventory_id	category	quantity_in_stock	warehouse_location	last_updated
INV001	Kaos	500	Jakarta	01/05/2025
INV002	Celana	300	Bandung	02/05/2025
INV003	Jaket	200	Jakarta	03/05/2025

2. Sales

sale_id	sale_date	category	sale_amount	revenue
SALE001	15/01/2025	Kaos	100	1500000
SALE002	10/02/2025	Celana	50	1000000
SALE003	05/03/2025	Jaket	20	800000

3. Production

production_id	category	quantity_produced	start_date	end_date
PROD001	Kaos	600	01/01/2025	10/01/2025
PROD002	Celana	400	01/02/2025	08/02/2025
PROD003	Jaket	250	01/03/2025	12/03/2025

Sesuai dengan urutan masing-masing 'task' diatas buat soal untuk kepentingan bisnis pabrikasi khususnya pabrik garment. Buat tabel datasetnya sesuai dengan kondisi urutan diatas. Tabel dibuat dengan 2 versi yaitu excel dan csv.

Berdasarkan urutan data analitik yang telah dijelaskan sebelumnya, saya akan membuat soal untuk kepentingan bisnis pabrik garment, merancang dataset yang relevan untuk analisis di Google BigQuery, serta menyediakan tabel dataset dalam format Excel dan CSV yang dapat diunduh. Dataset akan mencakup tabel **persediaan (inventory)**, **penjualan (sales)**, dan **produksi (production)**, yang disesuaikan dengan kebutuhan analitik pabrik garment.

Soal untuk Kepentingan Bisnis Pabrik Garment

Berikut adalah soal yang dirancang untuk setiap tahapan analitik, dengan fokus pada bisnis pabrik garment:

1. Define Objective (Menentukan Tujuan Analisis)

- **Soal:** Pabrik garment ingin mengetahui performa penjualan dan efisiensi produksi dalam 6 bulan terakhir untuk menentukan strategi pengelolaan stok dan perencanaan produksi. Buat kueri untuk menghitung:
 - a. Total penjualan per kategori produk (misalnya, kaos, celana, jaket).
 - b. Persentase produk terjual dari total produksi per kategori.
 - c. Rata-rata waktu penyelesaian produksi per batch.
- **Tujuan:** Membantu manajemen memahami produk yang laris, efisiensi produksi, dan kebutuhan stok.

2. Data Exploration (Eksplorasi Data)

- **Soal:** Jelajahi dataset penjualan, persediaan, dan produksi untuk mengidentifikasi:
 - a. Kolom mana yang memiliki nilai null?
 - b. Distribusi penjualan per kategori produk.
 - c. Apakah ada anomali, seperti penjualan negatif atau tanggal produksi di masa depan?
- **Tugas BigQuery:** Tulis kueri untuk memeriksa null values dan distribusi data, misalnya:

sql

Copy

```
SELECT category, COUNT(*) AS total_sales, COUNTIF(sale_amount IS
NULL) AS null_sales

FROM `project.garment.sales`

GROUP BY category;
```

3. Data Preparation (Persiapan Data)

- **Soal:** Siapkan data untuk analisis dengan:
 - a. Menggabungkan tabel penjualan dan persediaan untuk menghitung stok tersedia vs. terjual.
 - b. Membersihkan data, seperti mengonversi format tanggal dan menangani nilai null pada jumlah produksi.
 - c. Membuat CTE untuk menyaring data hanya untuk 6 bulan terakhir.
- **Tugas BigQuery:** Contoh kueri:

sql

Copy

```
WITH cleaned_sales AS (  
  
    SELECT sale_id, CAST(sale_date AS DATE) AS sale_date, category,  
    COALESCE(sale_amount, 0) AS sale_amount  
  
    FROM `project.garment.sales`  
  
    WHERE sale_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 6 MONTH)  
  
)  
  
SELECT * FROM cleaned_sales;
```

4. Query Development (Pengembangan Kueri)

- **Soal:** Buat kueri untuk menjawab tujuan awal:
 - a. Total penjualan per kategori produk.
 - b. Persentase produk terjual dari total produksi.
 - c. Rata-rata waktu produksi per batch.
- **Tugas BigQuery:** Contoh kueri:

sql

Copy

```
SELECT  
  
    s.category,  
  
    SUM(s.sale_amount) AS total_sales,  
  
    (SUM(s.sale_amount) / SUM(p.quantity_produced)) * 100 AS  
percent_sold,  
  
    AVG(TIMESTAMP_DIFF(p.end_date, p.start_date, DAY)) AS  
avg_production_days  
  
FROM `project.garment.sales` s  
  
JOIN `project.garment.production` p
```

```
ON s.category = p.category
```

```
WHERE s.sale_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 6 MONTH)
```

```
GROUP BY s.category;
```

5. Query Optimization (Optimasi Kueri)

- o **Soal:** Optimalkan kueri di atas agar lebih efisien dengan:
 - a. Menggunakan partisi pada tabel berdasarkan tanggal penjualan.
 - b. Memfilter data sebelum join untuk mengurangi pemrosesan.
 - c. Menggunakan APPROX_COUNT_DISTINCT untuk estimasi cepat jika diperlukan.
- o **Tugas BigQuery:** Contoh optimasi:

sql

Copy

```
SELECT
```

```
s.category,
```

```
SUM(s.sale_amount) AS total_sales
```

```
FROM `project.garment.sales` s
```

```
WHERE s.sale_date >= '2024-11-01' AND s.sale_date < '2025-05-01'
```

```
GROUP BY s.category;
```

6. Validation and Testing (Validasi dan Pengujian)

- o **Soal:** Validasi hasil kueri dengan:
 - a. Memeriksa apakah total penjualan sesuai dengan laporan manual.
 - b. Memastikan tidak ada duplikasi data penjualan.
 - c. Menguji kueri pada subset data (misalnya, hanya untuk bulan Januari 2025).
- o **Tugas BigQuery:** Contoh validasi:

sql

Copy

```
SELECT sale_id, COUNT(*) AS count
```

```
FROM `project.garment.sales`
```

```
GROUP BY sale_id
```

```
HAVING count > 1;
```

7. Visualization and Reporting (Visualisasi dan Pelaporan)

- **Soal:** Simpan hasil kueri sebagai tabel baru untuk laporan bulanan dan visualisasi di Google Data Studio. Jadwalkan kueri untuk pembaruan otomatis.
- **Tugas BigQuery:** Contoh:

sql

Copy

```
CREATE OR REPLACE TABLE `project.garment.sales_report`
```

```
AS
```

```
SELECT
```

```
category,
```

```
SUM(sale_amount) AS total_sales
```

```
FROM `project.garment.sales`
```

```
WHERE sale_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 6 MONTH)
```

```
GROUP BY category;
```

8. Iterate and Refine (Iterasi dan Penyempurnaan)

- **Soal:** Berdasarkan feedback, tambahkan dimensi seperti lokasi gudang pada kueri penjualan dan produksi untuk analisis lebih mendalam.
- **Tugas BigQuery:** Contoh kueri baru:

sql

Copy

```
SELECT
```

```

s.category,

i.warehouse_location,

SUM(s.sale_amount) AS total_sales

FROM `project.garment.sales` s

JOIN `project.garment.inventory` i

ON s.category = i.category

GROUP BY s.category, i.warehouse_location;

```

Desain Dataset

Dataset dirancang untuk mendukung analisis di atas, dengan tiga tabel: **inventory**, **sales**, dan **production**. Berikut adalah struktur tabel:

1. Tabel Inventory (Persediaan)

Column Name	Data Type	Description
inventory_id	STRING	ID unik untuk stok
category	STRING	Kategori produk (Kaos, Celana, Jaket)
quantity_in_stock	INT64	Jumlah stok tersedia
warehouse_location	STRING	Lokasi gudang (Jakarta, Bandung)
last_updated	DATE	Tanggal pembaruan stok

2. Tabel Sales (Penjualan)

Column Name	Data Type	Description
sale_id	STRING	ID unik untuk penjualan
sale_date	DATE	Tanggal penjualan
category	STRING	Kategori produk (Kaos, Celana, Jaket)
sale_amount	INT64	Jumlah produk terjual
revenue	FLOAT64	Pendapatan dari penjualan (Rp)

3. Tabel Production (Produksi)

Column Name	Data Type	Description
production_id	STRING	ID unik untuk batch produksi

Column Name	Data Type	Description
category	STRING	Kategori produk (Kaos, Celana, Jaket)
quantity_produced	INT64	Jumlah produk yang diproduksi
start_date	DATE	Tanggal mulai produksi
end_date	DATE	Tanggal selesai produksi

Contoh Data

Berikut adalah contoh data untuk masing-masing tabel dalam format CSV:

inventory.csv

inventory.csv
csv
Show inline

sales.csv

sales.csv
csv
Show inline

production.csv

production.csv
csv
Show inline

Format Excel

Untuk format Excel, data di atas dapat disimpan sebagai file .xlsx dengan struktur tabel yang sama. Karena Excel tidak dapat langsung di-embed sebagai artifact dalam respons ini, berikut adalah panduan untuk membuat file Excel:

1. Buat file Excel baru menggunakan perangkat lunak seperti Microsoft Excel atau Google Sheets.
2. Buat tiga sheet: Inventory, Sales, dan Production.
3. Salin data dari CSV di atas ke masing-masing sheet, pastikan header kolom sesuai dengan struktur tabel.
4. Simpan file sebagai garment_data.xlsx.

Catatan: File CSV di atas dapat diimpor ke Excel dengan mudah menggunakan fitur "Import from Text/CSV" di Excel atau Google Sheets.

Instruksi Pengunduhan

- **CSV:** Salin konten dari <xaiArtifact> di atas ke file teks dengan ekstensi .csv (misalnya, inventory.csv, sales.csv, production.csv) dan simpan di komputer Anda.
- **Excel:** Impor file CSV ke Excel atau masukkan data secara manual ke sheet Excel seperti dijelaskan di atas, lalu simpan sebagai .xlsx.

Catatan Tambahan

- Dataset ini dirancang untuk analisis di Google BigQuery. Pastikan tabel diunggah ke BigQuery dengan skema yang sesuai (misalnya, STRING untuk ID, DATE untuk tanggal, dll.).
- Untuk analisis lebih lanjut, gunakan kueri seperti yang ditunjukkan pada soal untuk menjawab pertanyaan bisnis.
- Jika Anda memerlukan kueri BigQuery spesifik atau dataset dengan lebih banyak data, silakan beri tahu saya!

Result di Google Cloud BigQuery

1. `SELECT category, COUNT(*) AS total_sales, COUNTIF(sale_amount IS NULL) AS null_sales
FROM `trans-dogfish-459611-n6.garment.sales`
GROUP BY category;`

✓ Query completed

Query results					Save results ▼	Open in ▼	
<	Job information	Results	Chart	JSON	Execution details	Export	>
Row	category ▼	total_sales ▼	null_sales ▼				
1	Kaos	1	0				
2	Celana	1	0				
3	Jaket	1	0				

✓ Query completed

Query results

Save results ▾

Open in ▾



Job information

Results

Chart

JSON

Execution details

Export



total_sales, null_sales by category

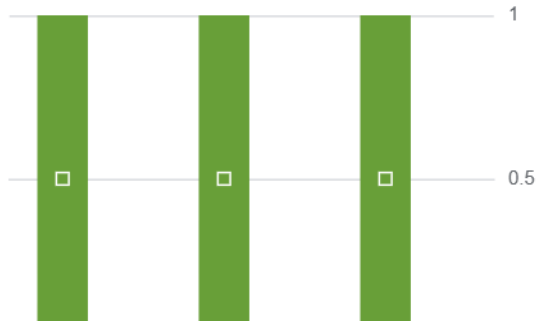


Chart configuration



Chart type

Bar



Dimension (x-axis)

category



Measures (y-axis)

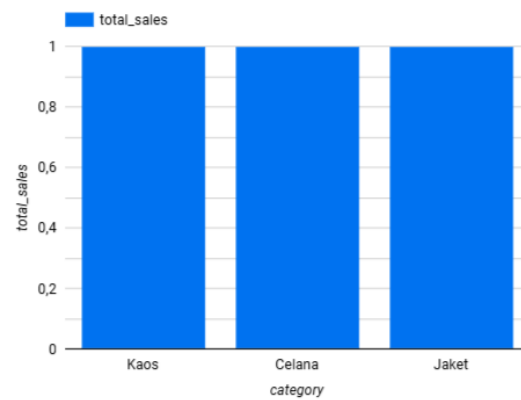
total_sales and null_sales



2.

sales

	category	total_sales ▾
1.	Kaos	1
2.	Celana	1
3.	Jaket	1



1 - 3 / 3 < >

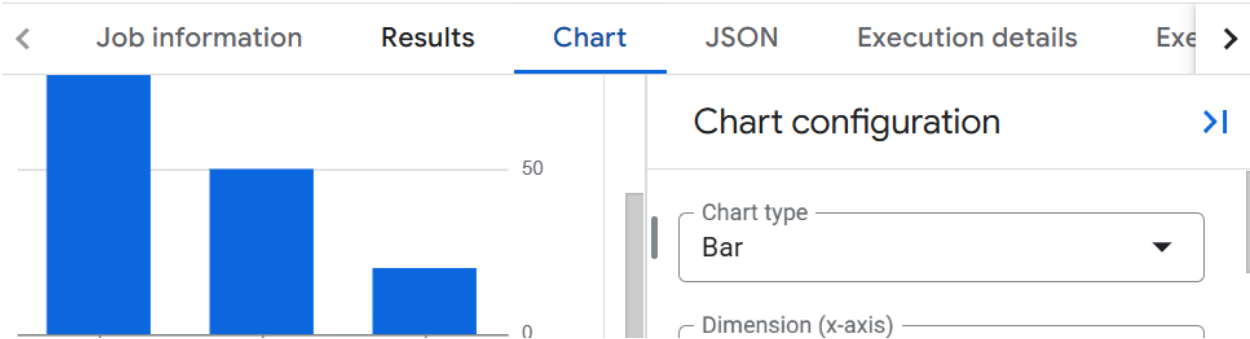

```
2. WITH cleaned_sales AS (  
    SELECT sale_id, CAST(sale_date AS DATE) AS sale_date, category,  
    COALESCE(sale_amount, 0) AS sale_amount  
    FROM `trans-dogfish-459611-n6.garment.sales`  
    WHERE sale_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 6 MONTH)  
)  
SELECT * FROM cleaned_sales;
```

< Job information Results Chart JSON Execution details Exe >					
Row	sale_id	sale_date	category	sale_ammoun	
1	SALE001	2025-01-15	Kaos		
2	SALE002	2025-02-10	Celana		
3	SALE003	2025-03-05	Jaket		

Query results

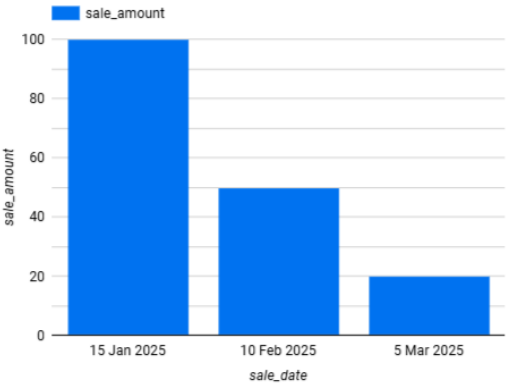
Save results

Open in



sales

	sale_id	sale_amount
1.	SALE001	100
2.	SALE002	50
3.	SALE003	20



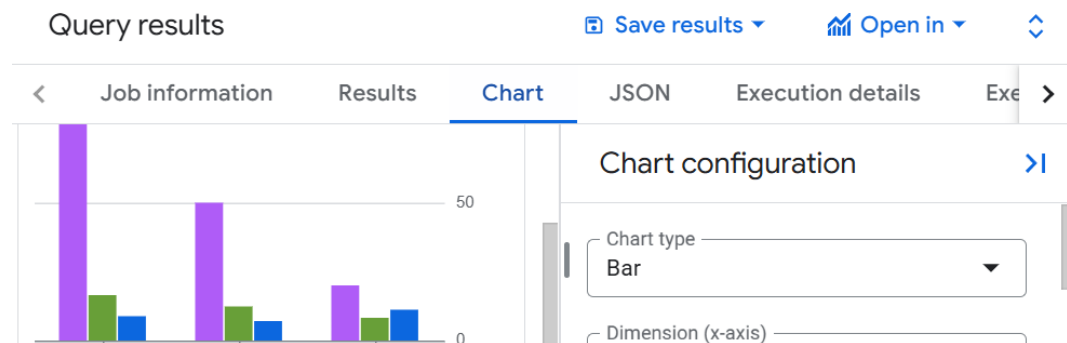
3. SELECT

```
s.category,  
SUM(s.sale_amount) AS total_sales,  
(SUM(s.sale_amount) / SUM(p.quantity_produced)) * 100 AS percent_sold,  
AVG(TIMESTAMP_DIFF(p.end_date, p.start_date, DAY)) AS avg_production_days  
FROM `trans-dogfish-459611-n6.garment.sales` s  
JOIN `trans-dogfish-459611-n6.garment.production` p  
ON s.category = p.category  
WHERE s.sale_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 6 MONTH)  
GROUP BY s.category;
```

Query results [Save results](#) [Open in](#)

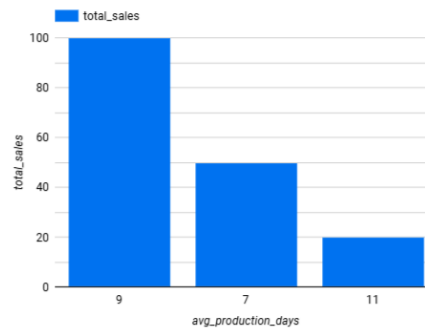
Row	category	total_sales	percent_sold	avg_production_days
1	Kaos	100	16.666666666666666...	9.0
2	Celana	50	12.5	7.0
3	Jaket	20	8.0	11.0

Query completed



sales

category	total_sales
1. Kaos	100
2. Celana	50
3. Jaket	20



```
4. SELECT
  s.category,
  SUM(s.sale_amount) AS total_sales
FROM `trans-dogfish-459611-n6.garment.sales` s
WHERE s.sale_date >= '2024-11-01' AND s.sale_date < '2025-05-01'
GROUP BY s.category;
```

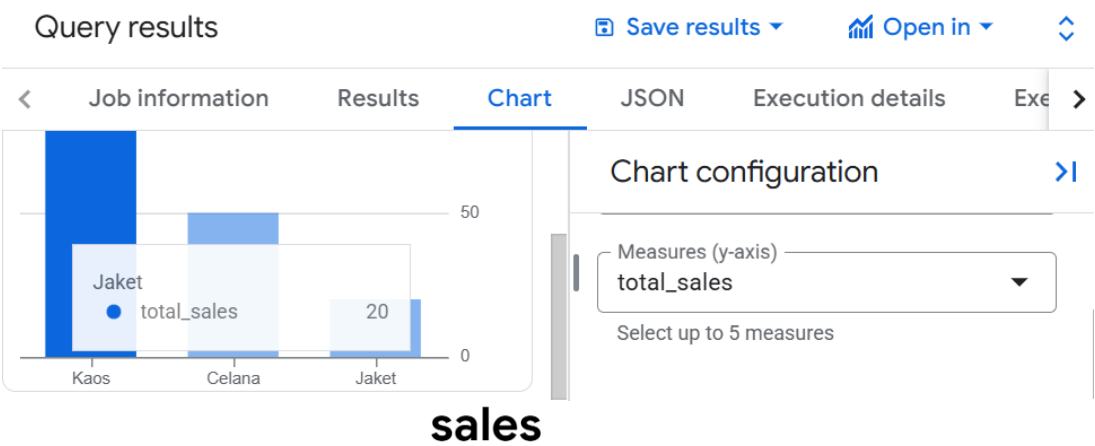
Query completed

Query results [Save results](#) [Open in](#)

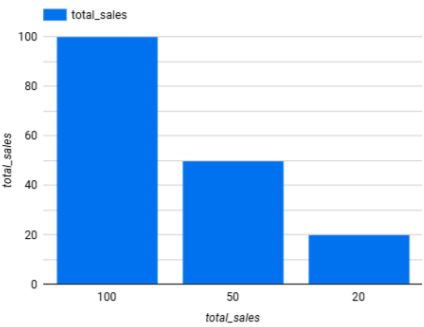
Job informationResultsChartJSONExecution details

Row	category	total_sales
1	Kaos	100
2	Celana	50
3	Jaket	20

Query completed



	category	total_sales
1.	Kaos	100
2.	Celana	50
3.	Jaket	20



```
5. SELECT sale_id, COUNT(*) AS count
FROM `trans-dogfish-459611-n6.garment.sales`
GROUP BY sale_id
HAVING count > 1;
```

The screenshot shows a SQL query editor with a tab labeled "Untitled query". The query text is as follows:

```
1 CREATE OR REPLACE TABLE `trans-dogfish-459611-n6.garment.sales`
2 AS
3 SELECT
4     category,
5     SUM(sale_amount) AS total_sales
6 FROM `trans-dogfish-459611-n6.garment.sales`
7 WHERE sale_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 6 MONTH)
8 GROUP BY category;
```

Below the query editor, a status bar indicates "Query completed" with a green checkmark icon.

The "Query results" section is visible, with tabs for "Job information", "Results" (selected), "Execution details", and "Execution graph".

A message at the bottom states: "This statement replaced the table named sales." with a "Go to table" button.

```
6. CREATE OR REPLACE TABLE `trans-dogfish-459611-n6.garment.sales`
AS
SELECT
    category,
    SUM(sale_amount) AS total_sales
FROM `trans-dogfish-459611-n6.garment.sales`
WHERE sale_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 6 MONTH)
GROUP BY category;
```

Explorer + Add data <

Search BigQuery resources ?

☐ Show starred only

- ▶ Data preparations
- ▶ Pipelines
- ▶ External connections
- ▶ farmasi
- ▼ garment
 - inventory
 - production
 - sales

[Show more](#)

Repository Preview ▼

no repository selected
Select a repository and a workspace to view its content.

[View repositories](#)

Untitled query Run Save Download

```

1 CREATE OR REPLACE TABLE `trans-dogfish-459611-n6.garment.sales`
2 AS
3 SELECT
4   category,
5   SUM(sale_amount) AS total_sales
6 FROM `trans-dogfish-459611-n6.garment.sales`
7 WHERE sale_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 6 MONTH)
8 GROUP BY category;

```

Query completed

Query results Save results Open in ↕

Job information Results Execution details Execution graph

i This statement replaced the table named sales. Go to table

Resource ID copied to clipboard. ×

[Refresh](#) ^

7. SELECT

```

s.category,
i.warehouse_location,
SUM(s.quantity_sold) AS total_sales
FROM `trans-dogfish-459611-n6.garment.sales` s
JOIN `trans-dogfish-459611-n6.garment.inventory` i
ON s.category = i.category
GROUP BY s.category, i.warehouse_location;

```

	Job information	Results	Chart	JSON	Execu
Row	category	total_sales			
1	Kaos	100			
2	Celana	50			
3	Jaket	20			